CS 203: Software Tools & Techniques for AI

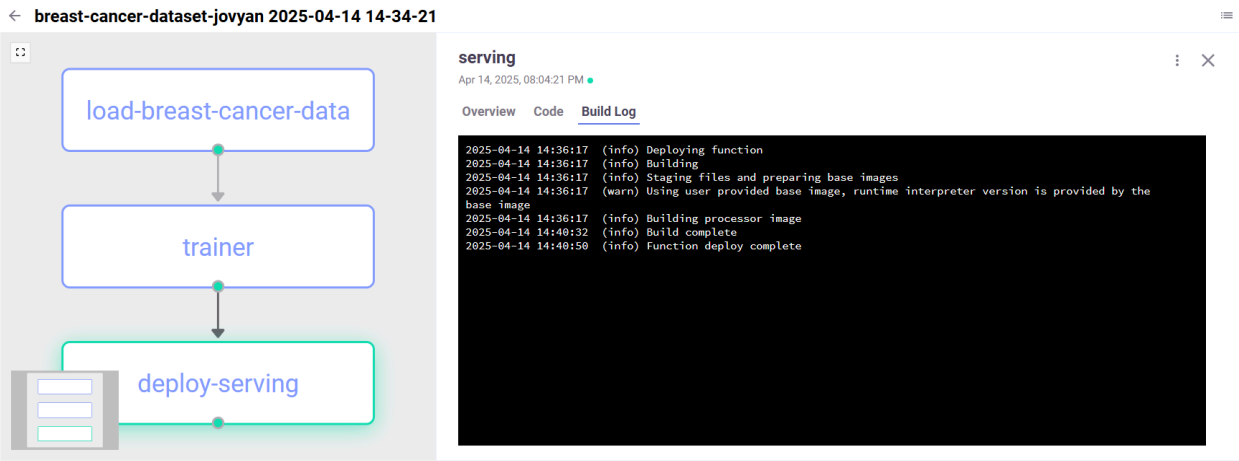
IIT Gandhinagar

Sem-II - 2024-25

Lab Assignment 9

CI/CD for Machine Learning using [MLRun],

Group -> 35
Memeber 1- Himanshu Singhal (24120030)
Member 2 - Gaurav Kumar (24120027)



1. **Create a mlrun project:**
   a. Use mlrun new_project. **[10%]**

```
13: helm --namespace mlrun install mlrun-ce --wait --timeout 1800s --set global.registry.url=index.docker.io/saileshpanda97 --set global.registry.secretName=registry-credentials --set kube-prometheus-stack.ena
14: helm uninstall mlrun-ce --namespace mlrun
15: helm --namespace mlrun install mlrun-ce --wait --timeout 1800s --set global.registry.url=index.docker.io/saileshpanda97 --set global.registry.secretName=registry-credentials --set kube-prometheus-stack.ena
16: helm uninstall mlrun-ce --namespace mlrun
17: helm --namespace mlrun install mlrun-ce --wait --timeout 1800s --set global.registry.url=index.docker.io/saileshpanda97 --set global.registry.secretName=registry-credentials --set kube-prometheus-stack.ena
```

```
D:\IITGN\Software Ai\Lab_9>helm --namespace mlrun install mlrun-ce --wait --timeout 1800s --set global.registry.url=index.docker.io/saileshpanda97 --set global.registry.secretName=registry-credentials --set kube
-prometheus-stack.enabled=false  mlrun-ce/mlrun-ce
NAME: mlrun-ce
LAST DEPLOYED: Fri Apr 11 11:53:34 2025
NAMESPACE: mlrun
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
You're up and running!

Jupyter UI is available at:
localhost:30040

Nuclio UI is available at:
localhost:30050

MLRun UI is available at:
localhost:30060

MLRun API is available at:
localhost:30070

Minio UI is available at:
localhost:30090
-  username: minio
-  password: minio123

Minio API is available at:
localhost:30080

Pipelines UI is available at:
localhost:30100

Happy MLOPSing!!! :]
```

1. **Create the following Python script:**
    a. **Data_prep.py: This fetches data using sklearn.datasets import load_breast_cancer. (Add screenshot of the code)[10 %]**

```python
import mlrun
from sklearn.datasets import load_breast_cancer
import pandas as pd

@mlrun.handler(outputs=["dataset", "label_column"])
def breast_cancer_loader(context, format="csv"):
    cancer = load_breast_cancer(as_frame=True)
    df = cancer.frame

    context.logger.info(f"Saving breast cancer dataset to {context.artifact_path}")
    context.log_dataset("breast_cancer_dataset", df=df, format=format, index=False)

    return df, "target"

if __name__ == "__main__":
    with mlrun.get_or_create_ctx("breast_cancer_generator", upload_artifacts=True) as context:
```

breast_cancer_loader(context, context.get_param("format", "csv"))
```

```python
import mlrun
from sklearn.datasets import load_breast_cancer
import pandas as pd

@mlrun.handler(outputs=["dataset", "label_column"])
def breast_cancer_loader(context, format="csv"):
    cancer = load_breast_cancer(as_frame=True)
    df = cancer.frame

    context.logger.info(f"Saving breast cancer dataset to {context.artifact_path}")
    context.log_dataset("breast_cancer_dataset", df=df, format=format, index=False)

    return df, "target"

if __name__ == "__main__":
    with mlrun.get_or_create_ctx("breast_cancer_generator", upload_artifacts=True) as context:
        breast_cancer_loader(context, context.get_param("format", "csv"))
```

b. Trainer.py: Split the data into train test (10% test data). Train a model using the training data. Use Random forest classifier. Wrap the model with apply_mlrun from mlrun.frameworks.sklearn. (Add screenshot of the code)[10 %]

```pyhton
import mlrun
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from mlrun.frameworks.sklearn import apply_mlrun

def train(
    dataset: mlrun.DataItem,
    label_column: str = 'target',
    n_estimators: int = 100,
    max_depth: int = 5,
    model_name: str = "breast_cancer_classifier"
):
    df = dataset.as_df()
    X = df.drop(label_column, axis=1)
    y = df[label_column]

    # Split into training and testing sets (90/10)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=42)
```
```

```
    model = RandomForestClassifier(n_estimators=n_estimators, max_depth=max_depth,
random_state=42)
    apply_mlrun(model=model, model_name=model_name, x_test=X_test, y_test=y_test)

    model.fit(X_train, y_train)
```

C. serving.py: Create a model class that will inherit from mlrun.serving.V2ModelServer, enabling automatic support for model lifecycle methods like load() and predict(). (Add screenshot of the code)[10 %]

```python
from cloudpickle import load
import numpy as np
from typing import List
import mlrun

class ClassifierModel(mlrun.serving.V2ModelServer):
    def load(self):
        model_file, _ = self.get_model('.pkl')
        self.model = load(open(model_file, 'rb'))

    def predict(self, body: dict) -> List:
        feats = np.asarray(body['inputs'])
        results = self.model.predict(feats)
        return results.tolist()
```

```python
from cloudpickle import load
import numpy as np
from typing import List
import mlrun

class ClassifierModel(mlrun.serving.V2ModelServer):
    def load(self):
        model_file, _ = self.get_model('.pkl')
        self.model = load(open(model_file, 'rb'))

    def predict(self, body: dict) -> List:
        feats = np.asarray(body['inputs'])
        results = self.model.predict(feats)
        return results.tolist()
```

load 1/3

D. workflow.py: Create a Python script that defines an MLRun pipeline using the @dsl.pipeline decorator. It should include the following steps:

```python
@dsl.pipeline(name="breast-cancer-pipeline")
def pipeline(model_name="breast_cancer_classifier"):
```

i. **Data Ingestion(Add screenshot of the code) [3 %]**

```python
    # Data ingestion step
    ingest = mlrun.run_function(
        "breast-cancer-loader",
        name="load-breast-cancer-data",
        params={"format": "pq"},
        outputs=["dataset"]
    )
```

ii. **Model Training: Experiment with different hyperparamters(n_estimators: [10, 100, 200], max_depth=[2, 5, 10]). Keep max accuracy as selector. (Add screenshot of the code)[4 %]**

```python
    # Model training step with hyperparameter tuning
    train = mlrun.run_function(
        "trainer",
        inputs={"dataset": ingest.outputs["dataset"]},
        hyperparams={
            "n_estimators": [10, 100, 200],
            "max_depth": [2, 5, 10]
        },
        selector="max.accuracy",
        outputs=["model"]
    )
```

iii. **Model Deployment: Deploy the model into the base kubernetes image(mlrun/mlrun) using mlrun.deploy_function (Add screenshot of the code)[3 %]**

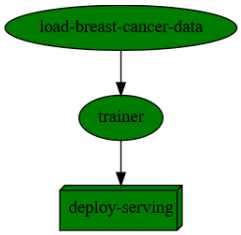
```python
# Model deployment step
deploy = mlrun.deploy_function(
    "serving",
    models=[{
        "key": model_name,
        "model_path": train.outputs["model"],
        "class_name": "ClassifierModel"
    }],
    mock=True
)
```

```python
import mlrun
from kfp import dsl

@dsl.pipeline(name="breast-cancer-pipeline")
def pipeline(model_name="breast_cancer_classifier"):

    # Data ingestion step
    ingest = mlrun.run_function(
        "breast-cancer-loader",
        name="load-breast-cancer-data",
        params={"format": "pq"},
        outputs=["dataset"]
    )

    # Model training step with hyperparameter tuning
    train = mlrun.run_function(
        "trainer",
        inputs={"dataset": ingest.outputs["dataset"]},
        hyperparams={
            "n_estimators": [10, 100, 200],
            "max_depth": [2, 5, 10]
        },
        selector="max.accuracy",
        outputs=["model"]
    )

    # Model deployment step
```

```
deploy = mlrun.deploy_function(
    "serving",
    models=[{
        "key": model_name,
        "model_path": train.outputs["model"],
        "class_name": "ClassifierModel"
    }],
    mock=True
)
```

3. Run the workflow using project.run:

a. **Add the screenshot of the workflow graph [10 %]**



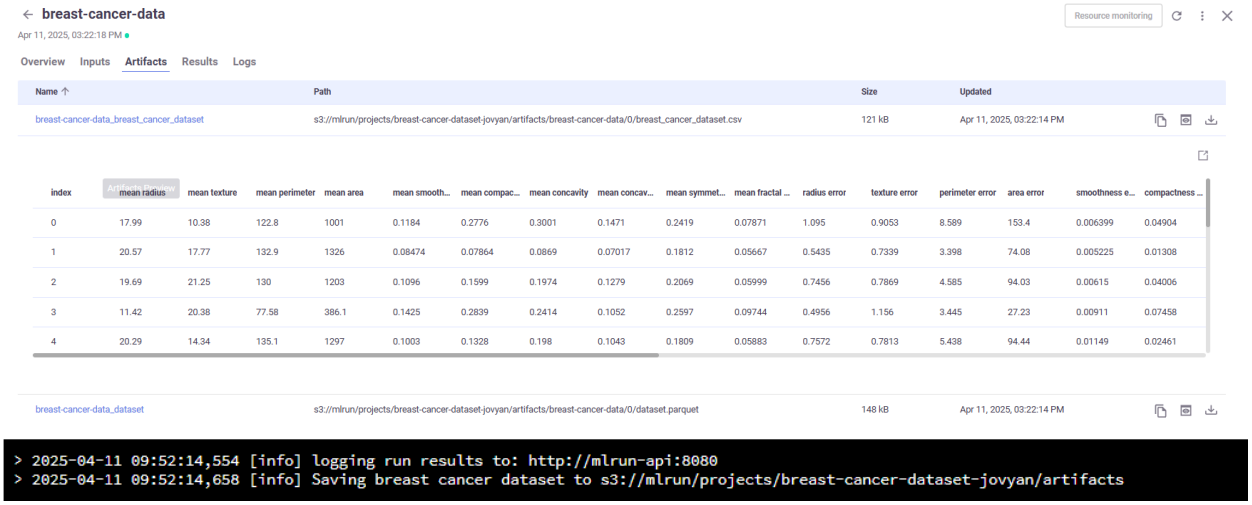
Run Results

[info] Workflow e8963042-e9c0-484e-808a-6c6265a7cdf1 finished, state=Succeeded

click the hyper links below to see detailed results

uid	start	state	kind	name	parameters	results
...b54a44e4	Apr 14 14:35:17	completed	run	trainer		best_iteration=1, accuracy=0.9649122807017544, f1_score=0.975, precision_score=0.975, recall_score=0.975
...bd6bf385	Apr 14 14:34:49	completed	run	load-breast-cancer-data	format=pq, model_name=breast-cancer-classifier	label_column=target

b. **Add the screenshot of the Data_prep artifact and take the screenshot of the dataframe. [10 %]**




```
> 2025-04-11 09:52:14,554 [info] logging run results to: http://mlrun-api:8080
> 2025-04-11 09:52:14,658 [info] Saving breast cancer dataset to s3://mlrun/projects/breast-cancer-dataset-jovyan/artifacts
```

code :

Loading dataset

```
gen_data_run = project.run_function("load-breast-cancer-data", params={"format":"csv"}, local=False)
```

```
> 2025-04-14 13:13:13,892 [warning] it is recommended to use k8s secret (specify secret_name), specifying the aws_access_key/aws_secret_key directly is unsafe
> 2025-04-14 13:13:13,930 [info] Storing function: {"db":"http://mlrun-api:8080","name":"load-breast-cancer-data","uid":"e1e38b1df6ad4918bd17bbe9c3e4bfca"}
> 2025-04-14 13:13:14,244 [info] Job is running in the background, pod: load-breast-cancer-data-l77n7
> 2025-04-14 13:15:53,208 [info] Handler was not provided running main (data_prep.py)
Running: ['/opt/conda/bin/python', '-u', 'data_prep.py']
> 2025-04-14 13:15:56,944 [info] logging run results to: http://mlrun-api:8080
> 2025-04-14 13:15:57,023 [info] Saving breast cancer dataset to s3://mlrun/projects/breast-cancer-dataset-jovyan/artifacts
> 2025-04-14 13:15:59,817 [info] To track results use the CLI: {"info_cmd":"mlrun get run e1e38b1df6ad4918bd17bbe9c3e4bfca -p breast-cancer-dataset-jovyan","logs_cmd":"mlrun logs e1e
38b1df6ad4918bd17bbe9c3e4bfca -p breast-cancer-dataset-jovyan"}
> 2025-04-14 13:15:59,818 [info] Run execution finished: {"name":"load-breast-cancer-data","status":"completed"}
```

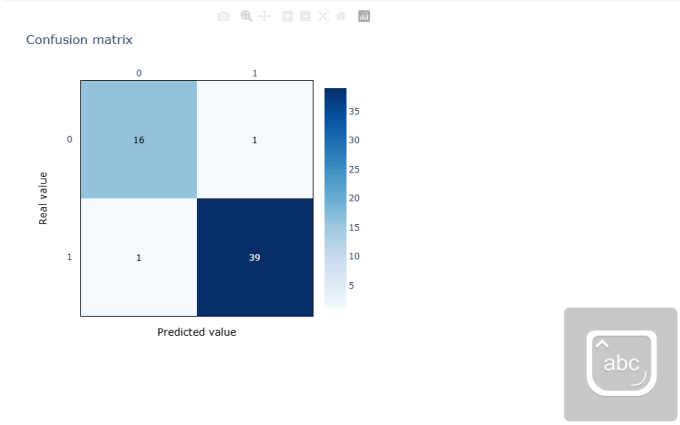
project	uid	iter	start	state	kind	name	labels	inputs	parameters	results	artifacts
breast-cancer-dataset-jovyan	...c3e4bfca	0	Apr 14 13:15:56	completed	run	load-breast-cancer-data	kind=job, owner=jovyan, mlrun/client_version=1.7.2, mlrun/client_python_version=3.9.13, host=load-breast-cancer-data-l77n7		format=csv	label_column=target	breast_cancer_dataset dataset

> **to track results use the .show() or .logs() methods or click here to open in UI**

```
> 2025-04-14 13:16:05,933 [info] Run execution finished: {"name":"load-breast-cancer-data","status":"completed"}
```

c. **Add the screenshot of the confusion-matrix artifact of train.py [10 %]**

Name	Path	Size	Updated
trainer_confusion-matrix	s3://mlrun/projects/breast-cancer-dataset-jovyan/artifacts/e8963042-e9c0-484e-808a-...	4.57 MB	Apr 14, 2025, 08:05:20 PM



d. **Add the screenshot of feature importance artifact. [10 %]**

Name	Path	Size	Updated
trainer_feature-importance	s3://mlrun/projects/breast-cancer-dataset-jovyan/artifacts/e8963042-e9c0-484e-808a-...	4.57 MB	Apr 14, 2025, 08:05:19 PM